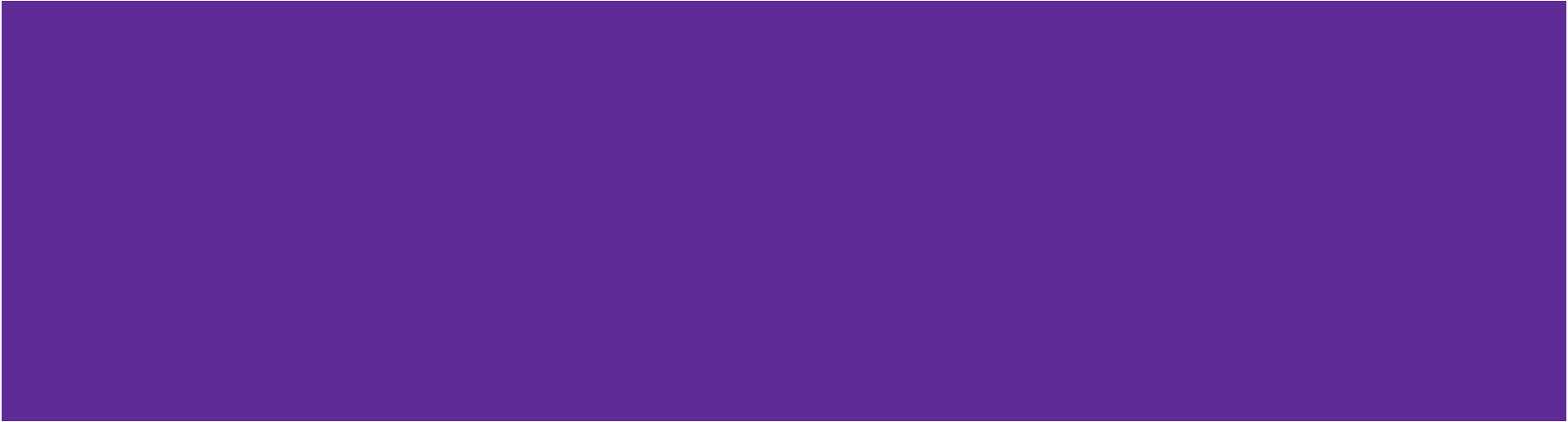


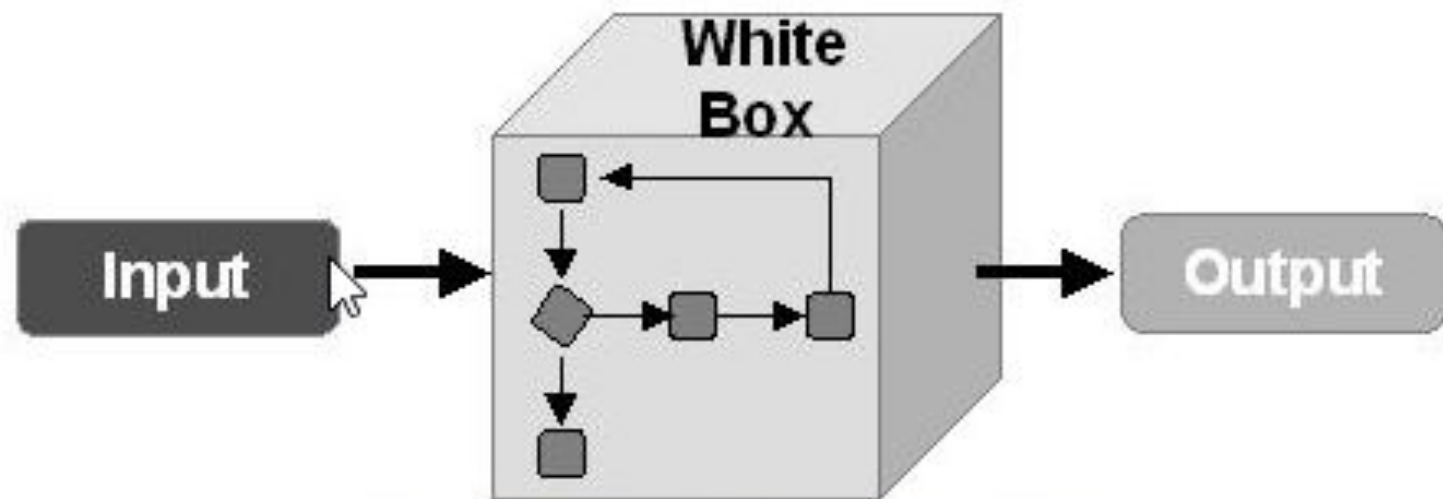
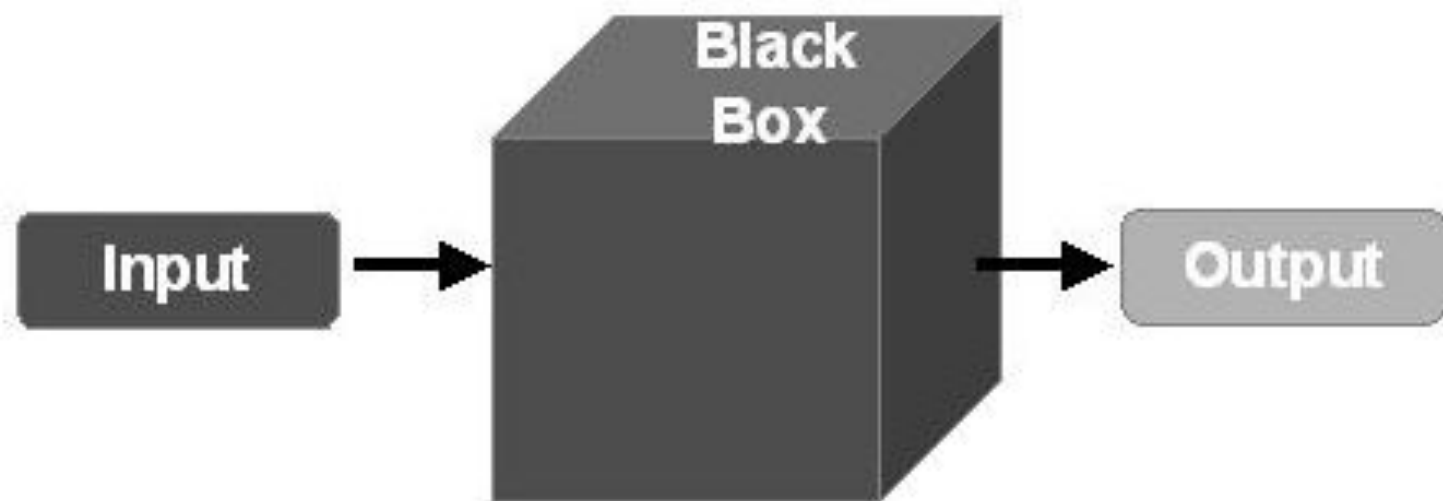
Outros testes

ESCOLA ESTADUAL DE ENSINO PROFISSIONAL
Prof. DAVI MAGALHÃES
QUALIDADE E TESTE DE SOFTWARE
AULA 04



Teste de Caixa Branca

O teste de software de caixa branca (também chamado de white-box testing, glass-box testing ou structural testing) é uma técnica de teste que se baseia no conhecimento **interno** do funcionamento do software.



Teste de Caixa Branca

O teste de software de caixa branca (também chamado de white-box testing, glass-box testing ou structural testing) é uma técnica de teste que se baseia no conhecimento **interno** do funcionamento do software. Diferente do teste de caixa preta, que se concentra nas entradas e saídas, o teste de caixa branca **examina o código-fonte, estruturas de controle de fluxo, condições lógicas e caminhos de execução do programa.**

Teste de Caixa Branca

Objetivo Principal:

- Garantir que **todas** as instruções, decisões, loops e condições do código estejam funcionando corretamente e tenham sido adequadamente testadas.

Teste de Caixa Branca

Características:

1. O testador precisa conhecer o código-fonte.
2. Verifica o funcionamento lógico e estrutural interno do programa.
3. Permite identificar:
4. Erros de lógica.
5. Caminhos não utilizados.
6. Condições não testadas.
7. Laços infinitos ou mal controlados.
8. Problemas de segurança, como falhas de autenticação.

Teste de Caixa Branca

Técnicas comuns:

1. **Cobertura de Instrução (Statement Coverage):** Verifica se todas as instruções do código foram executadas pelo menos uma vez.

Teste de Caixa Branca

Técnicas comuns:

1. **Cobertura de Instrução (Statement Coverage):** Verifica se todas as instruções do código foram executadas pelo menos uma vez.
2. **Cobertura de Decisão (Decision/Branch Coverage):** Garante que todas as decisões (if, else, switch...) tomem todas as direções possíveis.

Teste de Caixa Branca

Técnicas comuns:

1. **Cobertura de Instrução (Statement Coverage):** Verifica se todas as instruções do código foram executadas pelo menos uma vez.
2. **Cobertura de Decisão (Decision/Branch Coverage):** Garante que todas as decisões (if, else, switch...) tomem todas as direções possíveis.
3. **Cobertura de Condição (Condition Coverage):** Testa todas as condições booleanas possíveis, tanto verdadeiras quanto falsas.

Teste de Caixa Branca

Técnicas comuns:

1. **Cobertura de Instrução (Statement Coverage):** Verifica se todas as instruções do código foram executadas pelo menos uma vez.
2. **Cobertura de Decisão (Decision/Branch Coverage):** Garante que todas as decisões (if, else, switch...) tomem todas as direções possíveis.
3. **Cobertura de Condição (Condition Coverage):** Testa todas as condições booleanas possíveis, tanto verdadeiras quanto falsas.
4. **Cobertura de Caminho (Path Coverage):** Testa todos os caminhos possíveis de execução do programa.

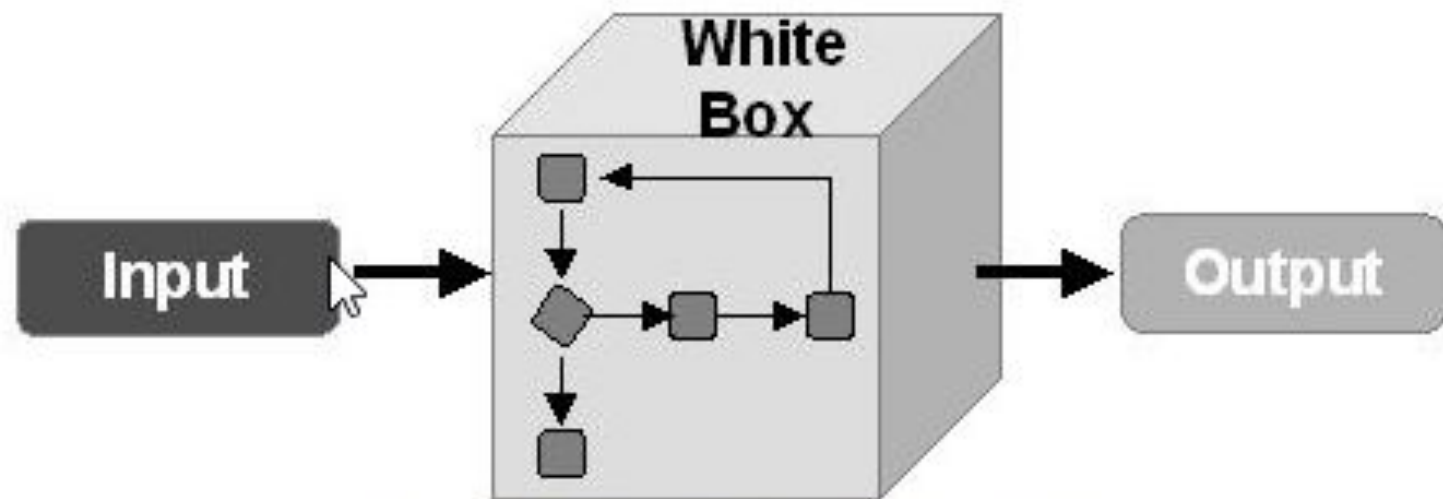
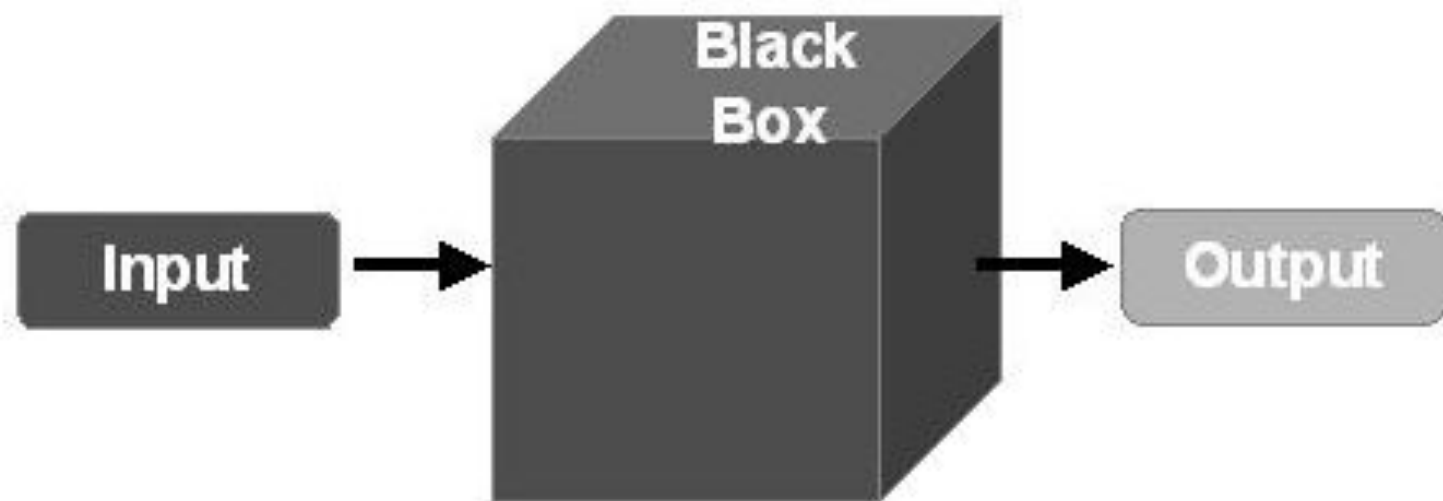
Teste de Caixa Branca

Técnicas comuns:

1. **Cobertura de Instrução (Statement Coverage):** Verifica se todas as instruções do código foram executadas pelo menos uma vez.
2. **Cobertura de Decisão (Decision/Branch Coverage):** Garante que todas as decisões (if, else, switch...) tomem todas as direções possíveis.
3. **Cobertura de Condição (Condition Coverage):** Testa todas as condições booleanas possíveis, tanto verdadeiras quanto falsas.
4. **Cobertura de Caminho (Path Coverage):** Testa todos os caminhos possíveis de execução do programa.
5. **Cobertura de Fluxo de Dados:** Verifica como os dados fluem pelo programa (declarações, usos, destruições de variáveis).

Teste de Caixa Preta

O teste de caixa preta (black-box testing) é uma técnica de teste de software que **foca no comportamento externo do sistema**, sem considerar seu funcionamento interno. O testador fornece entradas ao sistema e **observa as saídas** para verificar se elas estão de acordo com os requisitos especificados.



Teste de Caixa Preta

Objetivo Principal:

- Verificar se o software atende corretamente aos requisitos funcionais, testando entradas, saídas e interações com o sistema, sem conhecer o código-fonte.

Teste de Caixa Preta

Características:

1. O foco está na **funcionalidade** do sistema, conforme descrita nos requisitos.
2. **Não** se analisa o código nem a estrutura interna.
3. Baseia-se em casos de uso, requisitos do usuário, documentação funcional ou especificações.
4. Pode ser aplicado em testes de sistema, testes de aceitação, testes funcionais e testes de integração.

Teste de Caixa Preta

Técnicas comuns:

1. Particionamento de Equivalência: Divide as entradas em grupos que devem se comportar da mesma forma.

Teste de Caixa Preta

Técnicas comuns:

1. Particionamento de Equivalência: Divide as entradas em grupos que devem se comportar da mesma forma.
2. Análise de Valor Limite/Marco: Testa valores nas bordas de intervalos válidos e inválidos.

Teste de Caixa Preta

Técnicas comuns:

1. Particionamento de Equivalência: Divide as entradas em grupos que devem se comportar da mesma forma.
2. Análise de Valor Limite/Marco: Testa valores nas bordas de intervalos válidos e inválidos.
3. Tabela de Decisão: Usa combinações de condições para determinar ações esperadas.

Teste de Caixa Preta

Técnicas comuns:

1. Particionamento de Equivalência: Divide as entradas em grupos que devem se comportar da mesma forma.
2. Análise de Valor Limite/Marco: Testa valores nas bordas de intervalos válidos e inválidos.
3. Tabela de Decisão: Usa combinações de condições para determinar ações esperadas.
4. Transição de Estado: Testa mudanças de estado do sistema com base em eventos.

Teste de Caixa Preta

Técnicas comuns:

1. Particionamento de Equivalência: Divide as entradas em grupos que devem se comportar da mesma forma.
2. Análise de Valor Limite/Marco: Testa valores nas bordas de intervalos válidos e inválidos.
3. Tabela de Decisão: Usa combinações de condições para determinar ações esperadas.
4. Transição de Estado: Testa mudanças de estado do sistema com base em eventos.
5. Testes baseados em Casos de Uso: Valida os fluxos normais e alternativos descritos para os usuários.

Teste de Caixa Preta

Vantagens:

- Testes podem ser feitos por pessoas sem acesso ao código-fonte.

Teste de Caixa Preta

Vantagens:

- Testes podem ser feitos por pessoas sem acesso ao código-fonte.
- Foca nos requisitos do usuário, garantindo que o software tenha o comportamento esperado.

Teste de Caixa Preta

Vantagens:

- Testes podem ser feitos por pessoas sem acesso ao código-fonte.
- Foca nos requisitos do usuário, garantindo que o software tenha o comportamento esperado.
- Útil para testar sistemas complexos de forma independente da linguagem de programação.

Teste de Caixa Preta

Vantagens:

- Testes podem ser feitos por pessoas sem acesso ao código-fonte.
- Foca nos requisitos do usuário, garantindo que o software tenha o comportamento esperado.
- Útil para testar sistemas complexos de forma independente da linguagem de programação.
- Pode identificar falhas de especificação.

Teste de Caixa Preta

Desvantagens:

- Cobertura de teste pode ser incompleta, pois não considera o que não está documentado.

Teste de Caixa Preta

Desvantagens:

- Cobertura de teste pode ser incompleta, pois não considera o que não está documentado.
- Difícil identificar erros ocultos no código, como condições não executadas.

Teste de Caixa Preta

Desvantagens:

- Cobertura de teste pode ser incompleta, pois não considera o que não está documentado.
- Difícil identificar erros ocultos no código, como condições não executadas.
- Pode haver redundância de testes se os casos não forem bem planejados.

Teste de Caixa Preta

Desvantagens:

- Cobertura de teste pode ser incompleta, pois não considera o que não está documentado.
- Difícil identificar erros ocultos no código, como condições não executadas.
- Pode haver redundância de testes se os casos não forem bem planejados.
- Ineficiente para encontrar certos tipos de bugs internos (ex: lógica mal implementada).

Teste de Interface Gráfica

O teste de interface gráfica é um tipo de teste de software que verifica se a interface visual do sistema funciona corretamente e está de acordo com os requisitos. O objetivo é garantir que os elementos da interface estejam **presentes, bem posicionados, funcionais e que respondam corretamente às ações** do usuário.

Teste de Interface Gráfica

Objetivos:

- Verificar se todos os elementos gráficos (botões, campos, menus, imagens, janelas etc.) estão **visíveis e funcionando** corretamente.

Teste de Interface Gráfica

Objetivos:

- Verificar se todos os elementos gráficos (botões, campos, menus, imagens, janelas etc.) estão **visíveis e funcionando** corretamente.
- Avaliar se a navegação está **coerente e intuitiva**.

Teste de Interface Gráfica

Objetivos:

- Verificar se todos os elementos gráficos (botões, campos, menus, imagens, janelas etc.) estão **visíveis e funcionando** corretamente.
- Avaliar se a navegação está **coerente e intuitiva**.
- Checar se a interface segue padrões de **usabilidade** e **acessibilidade**.

Teste de Interface Gráfica

Objetivos:

- Verificar se todos os elementos gráficos (botões, campos, menus, imagens, janelas etc.) estão **visíveis e funcionando** corretamente.
- Avaliar se a navegação está **coerente e intuitiva**.
- Checar se a interface segue padrões de **usabilidade** e **acessibilidade**.
- Testar o comportamento da interface em **diferentes resoluções, dispositivos ou navegadores**.

Teste de Interface Gráfica

Botões e links:

- Estão funcionando?
- Executam a ação correta?
- Estão habilitados/desabilitados nos momentos certos?

Button:

Button

Submit

Reset

Input:

Button

Submit

Reset

[Link](#)

Button

`<a href="">`

`<button>`

Navigation

Action

Teste de Interface Gráfica

Campos de entrada (textos, senhas, caixas de seleção):

- Aceitam o tipo de dado esperado?
- Validam corretamente os dados inseridos?

Cadastro

Nome Email

Celular com DDD Hora desejada 

Seu site Senha

Data Nasc  Cor favorita

Qtde Filhos Peso 0 200

Sexo:

☐ Masculino

☐ Feminio

Estado 

Mensagem

Teste de Interface Gráfica

Mensagens de erro e validação:

- São exibidas corretamente quando há erro?
- Ajudam o usuário a entender o problema?



Email address

 *Incorrect format.*

✗ Don't!



Enter a valid email address to use as your Apple ID.

This will be your new Apple ID.

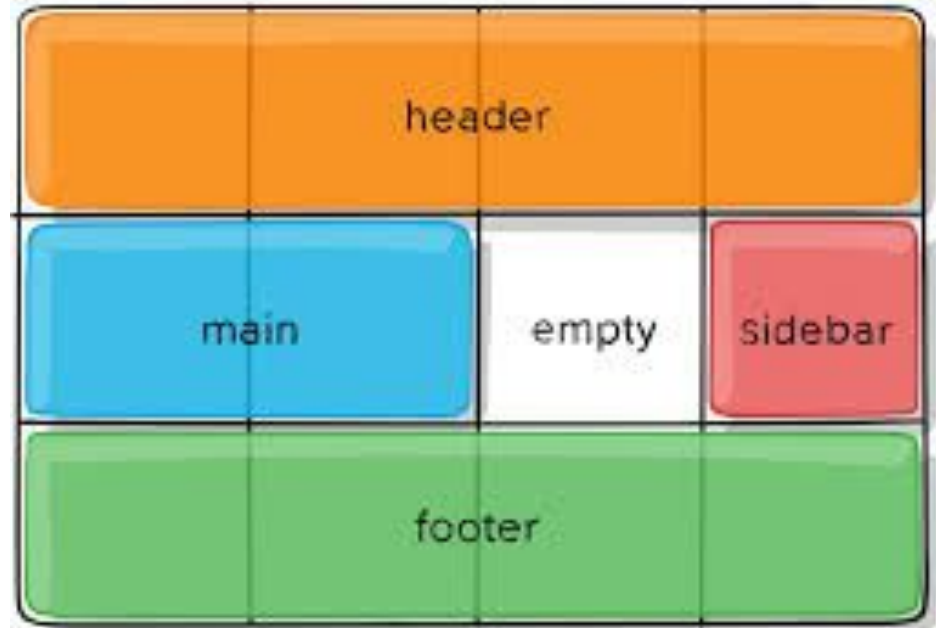
 Sorry, that password isn't right. We can help you [recover your password](#).

✓ Do

Teste de Interface Gráfica

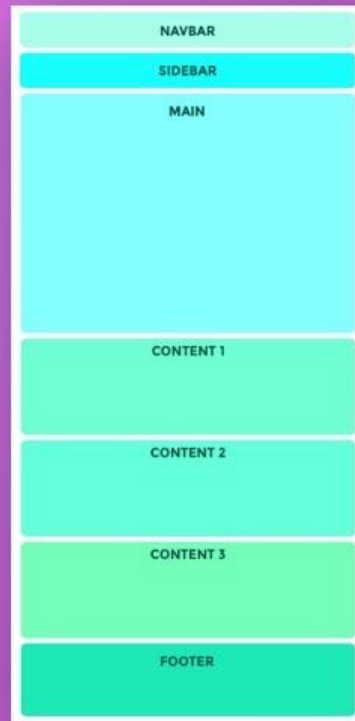
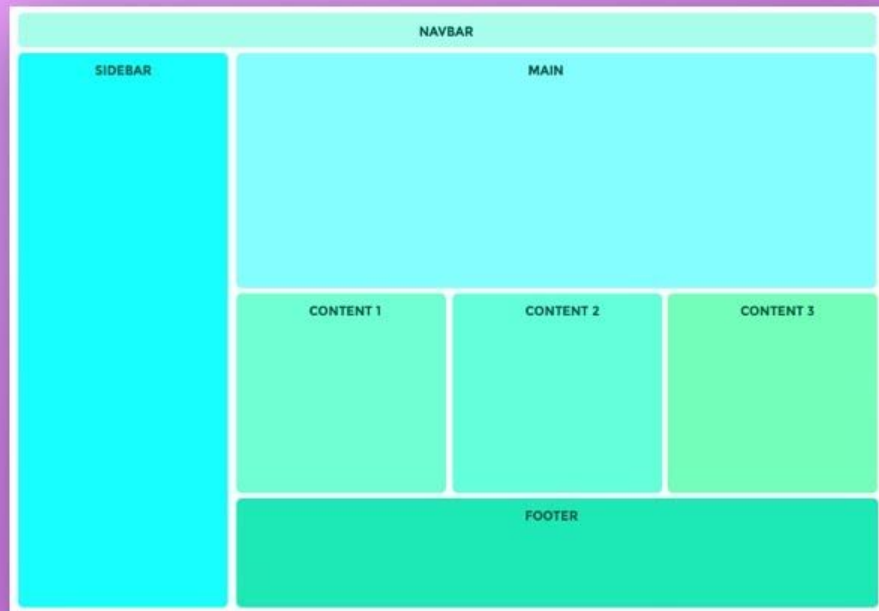
Layouts e design:

- Elementos estão alinhados?
- Cores e fontes seguem o padrão?
- A interface se adapta bem a diferentes telas?



RESPONSIVE CSS GRID

ONLY HTML & CSS



**WEB TO
MOBILE
DESIGN**

Teste de Interface Gráfica

Comportamentos dinâmicos:

- O que acontece ao clicar, arrastar, digitar ou passar o mouse?
- Há efeitos visuais esperados (ex: animações, feedbacks)?

Teste de Interface Gráfica

Vantagens:

- Garante uma boa experiência para o usuário final.

Teste de Interface Gráfica

Vantagens:

- Garante uma boa experiência para o usuário final.
- Ajuda a encontrar problemas visuais e de usabilidade que outros testes não detectam.

Teste de Interface Gráfica

Vantagens:

- Garante uma boa experiência para o usuário final.
- Ajuda a encontrar problemas visuais e de usabilidade que outros testes não detectam.
- Essencial para sistemas orientados à interação com o usuário (ex: aplicativos e sistemas web).

Teste de Interface Gráfica

Desvantagens:

- Testes manuais podem ser demorados e sujeitos a erros humanos.
- Testes automatizados de GUI podem quebrar com mudanças visuais simples, exigindo manutenção constante.
- Nem sempre detectam erros lógicos internos — por isso devem ser combinados com outros tipos de testes.

Contato

davimagal.sc@gmail.com